

RED NEURONAL PARA LA COMPUERTA AND

1.1 BASE TEORICA: Clasificación binaria y entropía cruzada.

Para la clasificación binaria la red neuronal predice si una entrada pertenece a una de las 2 clases.

0 → clase negativa

1 → clase positiva

La red no entrega un valor binario directamente sino una probabilidad $\hat{y}$ de que la entrada pertenezca a la clase 1.

$$\hat{y} = P(y=1|x)$$

Esta probabilidad se calcula mediante la función sigmoide

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

La pérdida binaria de entropía cruzada (BCE) mide la discrepancia entre la salida predicha $\hat{y}$ y el valor real y .

$$L(y, \hat{y}) = -(y \ln(y) + (1-y) \ln(1-\hat{y}))$$

1.2 ARQUITECTURA DE LA RED NEURONAL AND (2-2-1)

La red está diseñada para replicar el comportamiento de una compuerta AND.

Capa	Tamaño	Descripción
Entrada x	2	Dos valores binarios (0 o 1)
Capa oculta $a^{(1)}$	2	Cada neurona detecta una entrada
Salida $\hat{y}$	1	Resultado de la operación AND

1.2.1 Pesos y Bías Iniciales

Primera capa (entrada - oculta)

$$W^{(1)} = \begin{bmatrix} 10 & 0 \\ 0 & 10 \end{bmatrix} \quad b^{(1)} = \begin{bmatrix} -5 \\ -5 \end{bmatrix}$$

Segunda capa (oculta - salida)

$$W^{(2)} = \begin{bmatrix} 10 & 10 \end{bmatrix} \quad b^{(2)} = -15$$

1.2.2 Justificación de los valores grandes

La función sigmoide comprime cualquier valor real entre 0 y 1

$$\sigma(5) = 0.993 = 1$$

$$\sigma(-5) = 0.007 = 0$$

Usar pesos grandes permite que la sigmoide se comporte como un escalón, simulando el comportamiento binario AND.

1.3 FORWARD PASS (Propagación directa)

1.3.1 Ecuación

Capa oculta

$$z^{(1)} = W^{(1)}x + b^{(1)} \quad a^{(1)} = \sigma(z^{(1)})$$

Capa de salida

$$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)} \quad \hat{y} = \sigma(z^{(2)})$$

1.3.2 Cálculo de la capa oculta

$$\text{Neurona 1 } z_1 = 10x_1 + 0x_2 - 5$$

$$\text{Neurona 2 } z_2 = 0x_1 + 10x_2 - 5$$

1.3.3 Cálculo de la capa salida

$$z^{(2)} = 10a_1^{(1)} + 10a_2^{(1)} - 15$$

1.4 FORWARD PASS para todas las entradas

Entrada x	$z^{(1)}$	$a^{(1)}$	$z^{(2)}$	$\hat{y}$	Salida
(0,0)	(-5, -5)	(0.006693, 0.006693)	-14.66	0.0000033	0
(0,1)	(-5, 5)	(0.006693, 0.993307)	-5.00	0.006693	0
(1,0)	(5, -5)	(0.993307, 0.006693)	-5.00	0.006693	0
(1,1)	(5, 5)	(0.993307, 0.993307)	4.66	0.993307	1

1.5 CÁLCULO DE PERDIDA CBCE

Entrada x	y	$\hat{y}$	BCE Loss
(0,0)	0	0.0000033	6.0000033
(0,1)	0	0.006693	6.006713
(1,0)	0	0.006693	6.006713
(1,1)	1	0.993307	0.006713

1.6 BACK PROPAGATION Paso a Paso

1.6.1 Error Capa Salida

$$\delta^{(2)} = \hat{y} - y = 0.992356 - 1 = -0.007644$$

1.6.2 Gradiente Capa Salida

$$\frac{\partial L}{\partial w^{(2)}} = \delta^{(1)} \cdot a^{(1)} = (-0.007593, -0.007593)$$

$$\frac{\partial L}{\partial b^{(1)}} = \delta^{(2)} = -0.007644$$

1.6.3 Propagation error a la capa oculta

$$\delta A^{(1)} = (W^{(2)})^T \cdot \delta^{(2)} = (-0.007644, -0.007644)$$

$$\sigma'(z^{(1)}) = a^{(1)} \odot (1 - a^{(1)}) = (0.006648, 0.006648)$$

$$\delta^{(1)} = \delta A^{(1)} \odot \sigma'(z^{(1)}) = (-0.000508, -0.000508)$$

1.6.4 Gradientes de la capa oculta

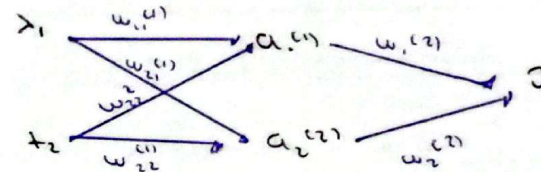
$$\frac{\partial L}{\partial w^{(1)}} = \delta^{(1)} \cdot x^T = \begin{bmatrix} -0.000508 & -0.000508 \\ -0.000508 & -0.000508 \end{bmatrix}, \frac{\partial L}{\partial b^{(1)}} = \delta^{(1)} = (-0.000508, -0.000508)$$

1.7 ACTUALIZACIÓN DE PESOS

$$w = w - \eta \cdot \nabla w$$

Parámetro	Antes	Cambio	Después
$w^{(2)}$	(10, 10)	-0.1 * (-0.007644)	(10.000754, 10.000754)
$b^{(2)}$	-15	-0.1 * (-0.007644)	-14.999236
$w^{(1)}$	(0, 0), (0, 10)	4000 00008	(0.000008, 0.000008)
$b^{(1)}$	(-5, -5)	+0.0000008	(-4.9999992, -4.9999992)

1.8 GRAFICO RED NEURONAL AND



2) RED NEURONAL COMPUENTA XOR

2.1 ARQUITECTURA (2-2-1)

Se utiliza la misma estructura pero los pesos y bias se ajustan para representar XOR. Primera capa Centrada-salida

$$w^{(1)} = \begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix}, b^{(1)} = \begin{bmatrix} -5 \\ -15 \end{bmatrix}$$

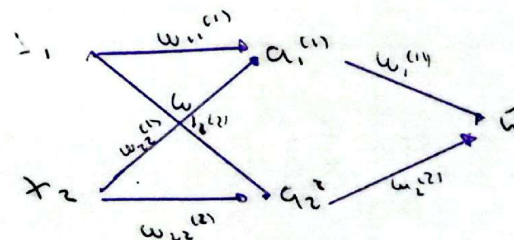
Segunda capa (oculta -> salida)

$$w^{(2)} = [10, 10], b^{(2)} = -5$$

2.2 FORWARD PASS, BACKPROPAGATION, PERDIDA Y ACTUALIZACIÓN DE PESOS

El procedimiento es dentro al AND pero los valores cambian para reflejar la operación XOR. Se calcula $z^{(1)}, a^{(1)}, z^{(2)}, \hat{y}$ luego BCE y gradientes $\delta^{(2)}, \delta^{(1)}$ con tasas de aprendizaje η y actualización de pesos δb .

2.3 GRAFICO DE LA RED XOR



3. RETOS DE REDES NEURONALES

- Vanishing / Exploding Gradients: en redes profundas, los gradientes pueden acercarse o crecer demasiado, dificultando el aprendizaje.
- Overfitting: redes grandes pueden memorizar los datos de entrenamiento perdiendo capacidad de generalización.
- Costo computacional: Calcular forward y backpropagation para redes grandes puede ser intensivo.

4. DESCENSO DE GRADIENTE ESTOCÁSTICO

El descenso de gradiente Estocástico (SGD) es un algoritmo fundamental para entrenar redes neuronales, su objetivo es minimizar la función de pérdida $L(y, \hat{y})$ ajustando los pesos y bias de la red.

4.1 Idea General

Calculamos la función de pérdida L en base a la salida predicha $\hat{y}$ y la salida real y . 2. Calculamos el gradiente de la pérdida con respecto a cada peso y bias.

$$\frac{\partial L}{\partial w} \text{ y } \frac{\partial L}{\partial b}$$

3. Ajustamos los pesos y bias en la dirección opuesta al gradiente

$$w \leftarrow w - \eta \frac{\partial L}{\partial w}, \quad b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

donde η es la tasa de aprendizaje que controla la magnitud del ajuste.

4.2 GRADIENTE

El gradiente indica la dirección de máximo incremento de la función de pérdida. Al moverse en la dirección opuesta al gradiente, disminuimos la pérdida y la red aprende a predecir mejor.

4.3 VARIANTES

- SGD puro actualiza los pesos después de cada ejemplo.
- Mini batch SGD calcula el gradiente usando un conjunto pequeño de ejemplos (mini-batch). Reduce la varianza y mejora la eficiencia computacional.
- Batch gradient descent calcula la gradiente sobre todo el conjunto de entrenamiento. Es más estable pero lento para grandes datasets.

4.4 ROL DE LA TASA DE APRENDIZAJE

- η demasiado grande: riesgo de saltar el mínimo o no converger.
- η demasiado pequeña: aprendizaje lento.
- Estrategias:
 - Decay de tasa de aprendizaje: reducir η gradualmente durante el entrenamiento.
 - Optimizadores adaptativos: como Adam, RMSprop que ajustan la tasa automáticamente para cada parámetro.

4.5 APLICACIÓN EN REDES NEURONALES

Realizaremos un forward pass para calcular $\hat{y}$, 2. calculamos la pérdida $L(y, \hat{y})$. 3. Hacemos Backpropagation para obtener los gradientes de cada peso y bias. 4. Aplicaremos la actualización usando SGD. Repetimos para todas las entradas (epocas) hasta que la pérdida converja.

4.6 VENTAJAS Y RETOS

- Permite entrenar redes grandes con eficiencia computacional razonable.
- Es sensible a la elección de η y al escalado de los datos.
- Puede quedar atrapado en mínimos locales, por eso se usan estrategias como momentum u optimizadores adaptativos.